



Haute Ecole de la Province de Liège

Techniques avancées de gestion des données

Bases de données avancées et XML

Leçon 11 : Bases de données exotiques

Samuel Hiard

Année académique 2023-2024

Table des matières

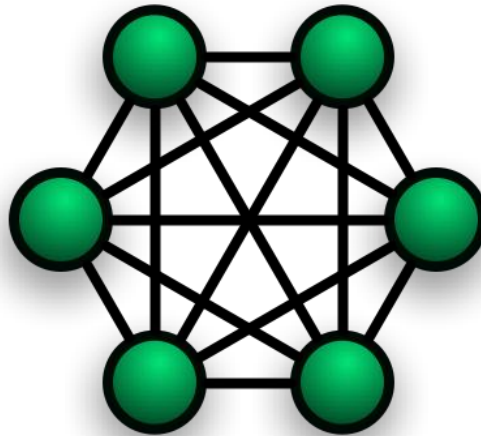
- Blockchain
- Bdd orientées objet
- NoSQL

Table des matières

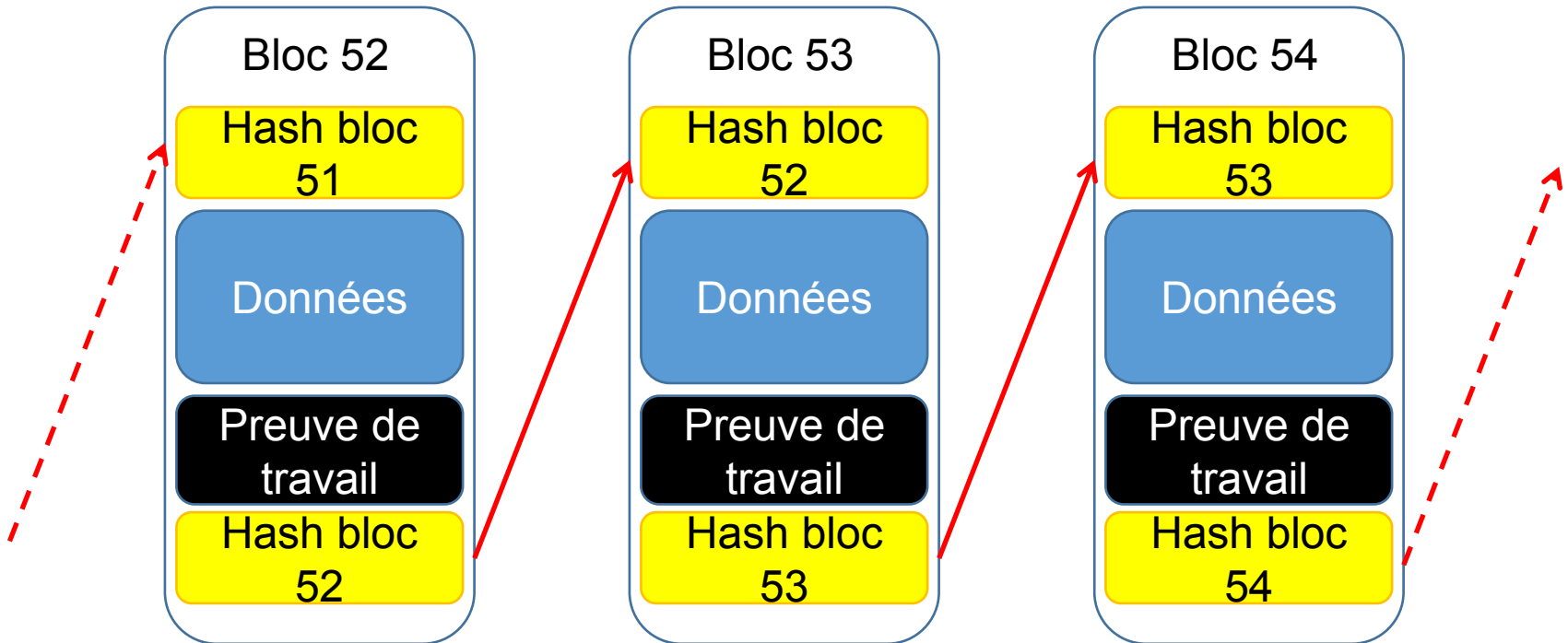
- Blockchain
- Bdd orientées objet
- NoSQL

Principe du blockchain

- Stockage de données numérique
 - Pérenne
 - Les données restent de manière permanente, ce qui peut aussi poser un problème de taille de données
 - Infalsifiable
 - Ou, du moins, très difficile à falsifier
 - Distribué
 - On parle plutôt de réseau maillé



Contenu d'une blockchain



Hash

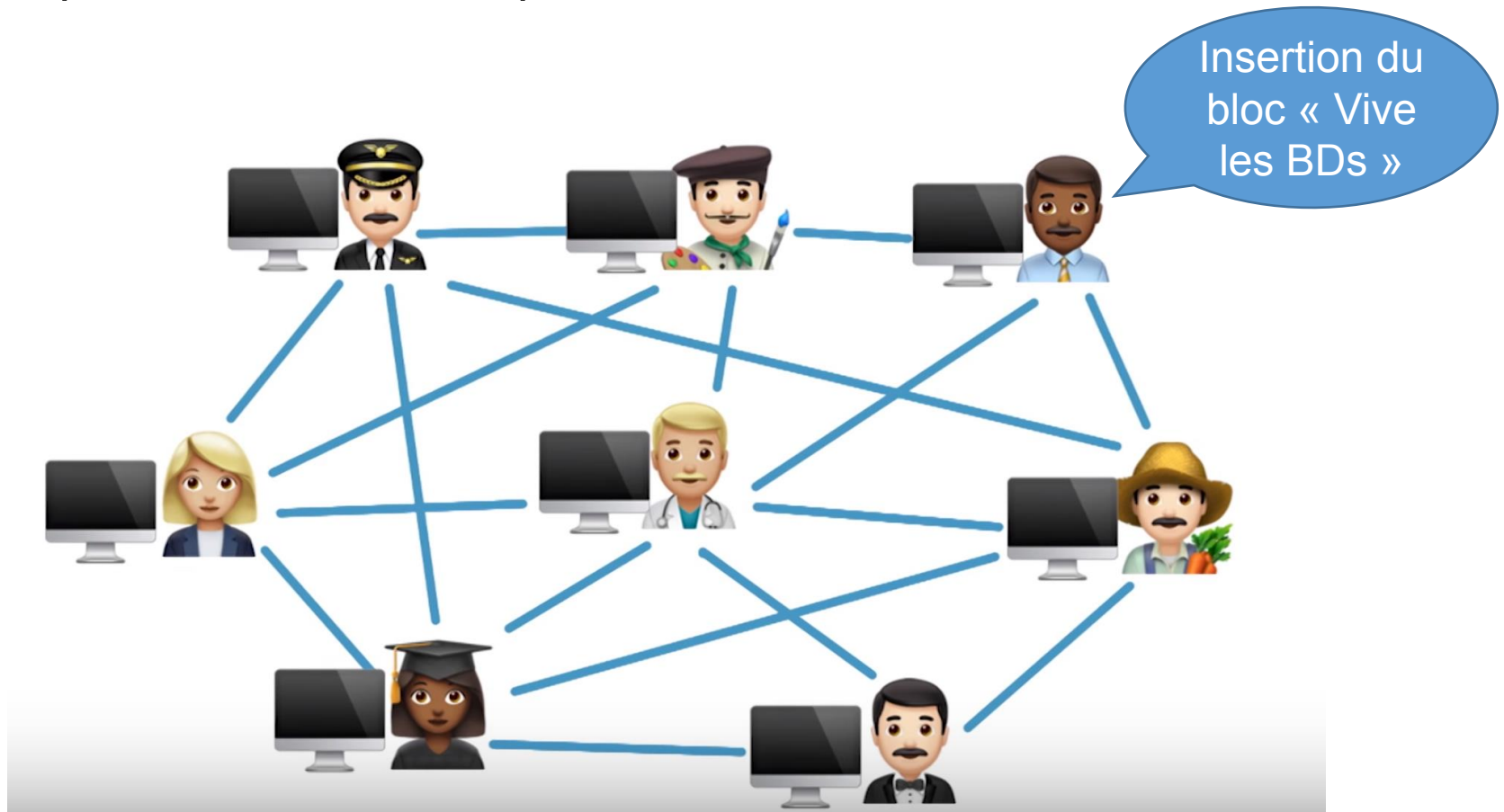
- Résultat de la fonction de hashage SHA-256 sur l'ensemble du bloc (hash du bloc précédent compris)
- 2 propriétés intéressantes :
 - Un petit changement de valeur d'entrée implique un code de sortie complètement différent
 - SHA-256(« J'aime les bases de données ») =
D86D1CAFC662F8762C18B6FD821306694DF49087B5DD86D37EE3788E0483AC7D
 - SHA-256(« J'aime les bases de données! »)=
11C181F698361C0ED0FB45A18E531421D37D35F8C041A430B8C6C1636AAAC786
 - Collisions possibles
 - Mais aucune trouvée encore à ce jour

Difficulté

- Chaque personne possède une version de la blockchain
- N'importe qui peut rajouter un bloc
 - Cela peut très vite devenir chaotique
- On ajoute la notion de difficulté : un bloc ne peut être ajouté que si son hash est inférieur à une certaine valeur (nommée « cible » ou « target »).
- C'est en modifiant la preuve de travail que l'on peut essayer d'obtenir un hash suffisamment faible.
- Il faut donc un grand nombre d'essais pour y arriver

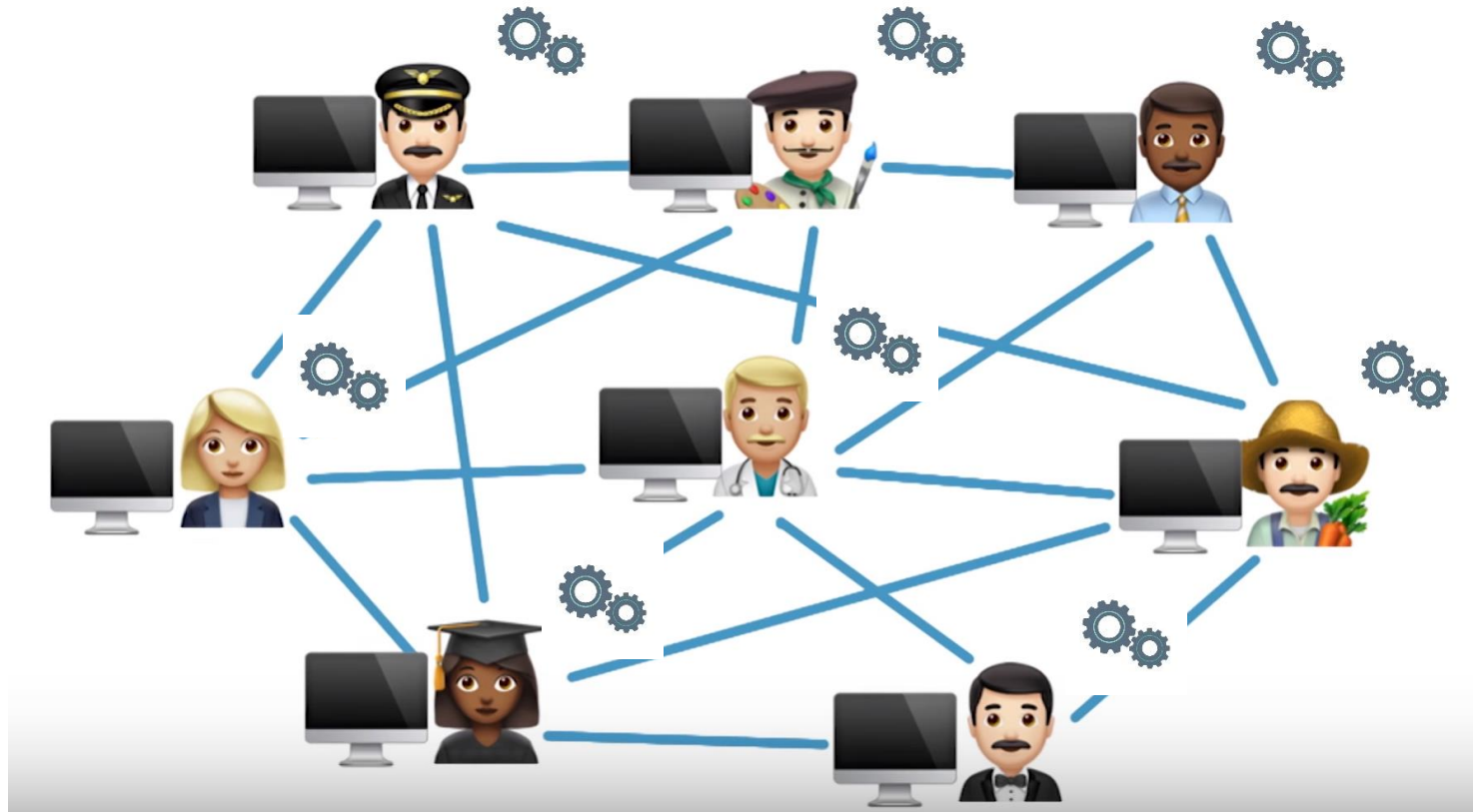
Ajout d'un bloc

- Etape 1 : envoi de la requête à tout le monde



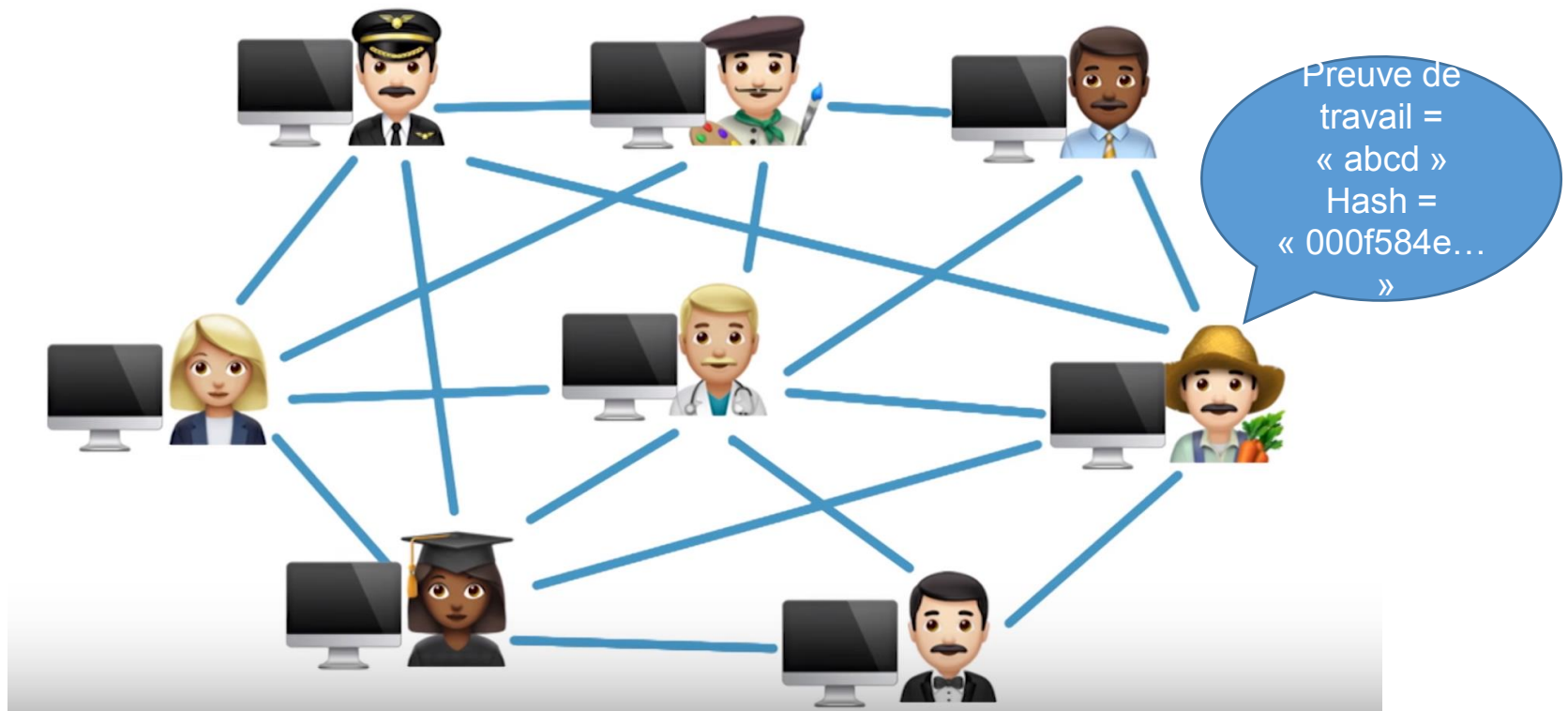
Ajout d'un bloc

- Etape 2 : calcul d'un hash inférieur à la cible



Ajout d'un bloc

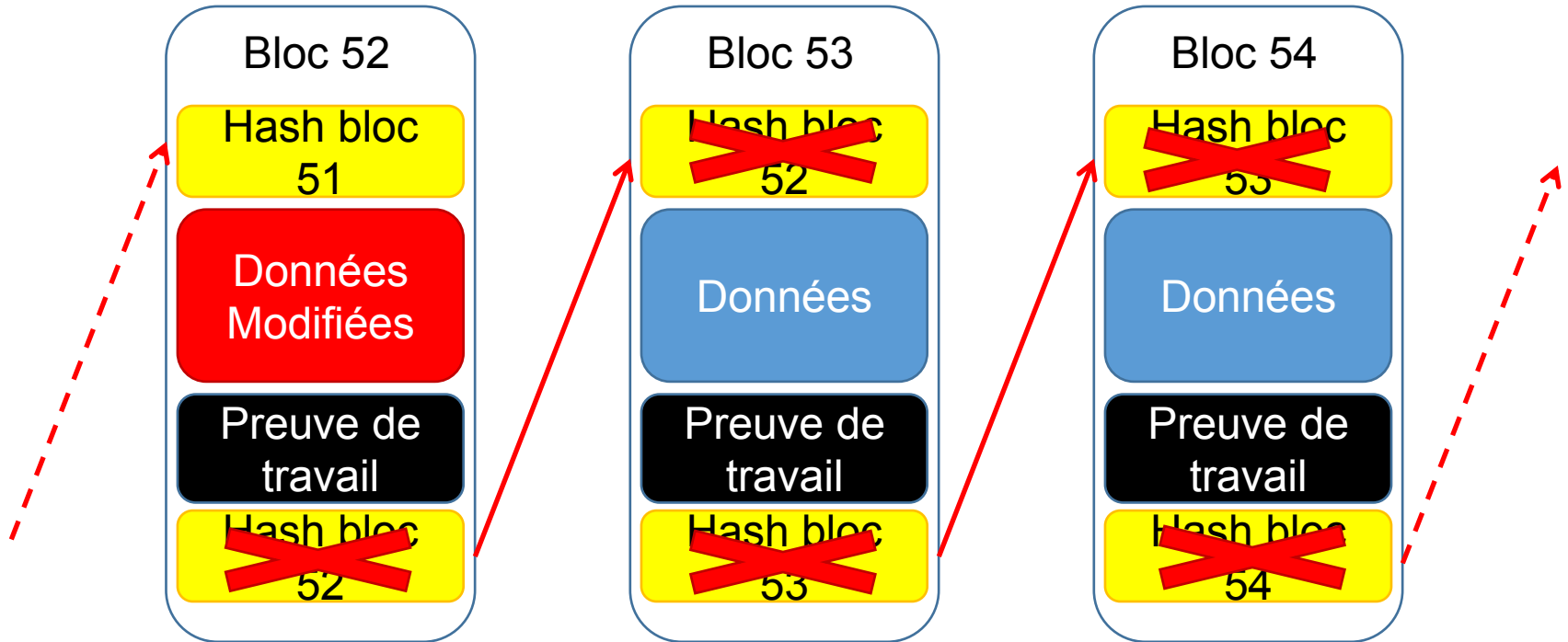
- Etape 3 : Diffusion du hash trouvé et ajout du bloc



Duplicata

- Il n'est pas impossible que deux personnes trouvent des preuves de travail différentes, mais avec un hash valide.
- La chaine aurait donc deux branches.
- Avec le temps, une branche sera plus utilisée que l'autre, conservation de la plus longue.

Sécurité



- Admettons que je veuille modifier les données du bloc 52
 - Le hash est donc invalide
 - Mais ce hash intervient aussi dans le bloc 53
 - Le hash du bloc 53 sera alors invalide, et ainsi de suite
 - Je dois donc recalculer l'ensemble des hashes à partir du bloc modifié et, en pratique, avoir plus de puissance de calcul que l'ensemble des participants
 - Le dernier bloc pourrait être falsifié, mais plus on remonte loin dans la chaîne, plus il sera difficile de falsifier un bloc

Validation, consensus et confirmation

- Validation : Etape pendant laquelle le contenu du bloc est vérifié à la recherche d'incohérences
 - Peut être fait avant ou après l'insertion du bloc
 - Doit être fait au moins une fois avant la phase suivante :
- Consensus : Etape de sélection (via un algorithme ou par compétition) du nœud qui pourra ajouter le bloc suivant
- Confirmation : Nombre de blocs nécessaires entre un bloc confirmé et le dernier bloc pour que l'on considère la probabilité de fork tellement faible que le bloc confirmé puisse être considéré comme pérenne.

Cryptomonnaies

- Blockchains stockant des informations monétaires
- Les deux plus connues : Bitcoin et Ethereum
- Problèmes à résoudre:
 - Que faire en cas de duplicata et comment les limiter?
 - Comment inciter les gens à calculer les hash des blocs?
 - Comment garantir la sécurité?

Limitation des duplicatas

- La difficulté de hashage est telle qu'il faut, en moyenne, environ 10 minutes pour qu'un nouveau bloc soit trouvé.
- Duplicatas toujours possibles, mais peu fréquents.
 - Dans tous les cas, le principe de base s'applique : la chaîne la plus longue est conservée.
- Comment garantir ce temps moyen de 10 minutes?
 - Si je mets en production un gros cluster, j'apporte plus de capacité de calcul mondialement et le temps moyen sera inférieur à 10 minutes.
- Solution : Modification de la difficulté du hashage tous les 2016 blocs ajoutés.

Sécurité du contenu

- Le monde entier ne peut générer, en moyenne, qu'un bloc toutes les 10 minutes.
 - Falsifier le dernier bloc impliquerait d'avoir plus de puissance de calcul que l'ensemble de la planète...
 - ... ou d'avoir de la chance.
- Le bloc est considéré comme valide lorsque la majorité des participants le déclarent comme tel
 - ! « Attaque des 51% » : une société possède >51% de tous les nœuds et décide de valider un « faux » bloc
- D'autre part, le contenu de chaque bloc est un ensemble de transactions (env. 2000) de type « *A* donne *X* bitcoins à *B* ».
 - Si *A* a rédigé ce message, c'est parfait
 - Si, c'est *B*, c'est problématique.

Sécurité du contenu

- Le monde entier ne peut générer, en moyenne, qu'un bloc toutes les 10 minutes.
- Falsifier le dernier bloc impliquerait d'avoir plus de puissance de calcul que l'ensemble de la planète...
- ... ou d'avoir de la chance.
- D'autre part, le contenu de chaque bloc est un ensemble de transactions (env. 2000) de type « *A* donne *X* bitcoins à *B* ».
 - Si *A* a rédigé ce message, c'est parfait
 - Si, c'est *B*, c'est problématique.

Cryptographie

- Chaque utilisateur possède deux clés cryptographiques
 - Une clé privée et une clé publique
- La transaction « A donne X bitcoins à B » est signée par A .
 - A utilise sa clé privée pour chiffrer un hash du message.
- Tout utilisateur peut dès lors utiliser la clé publique de A pour vérifier si le message n'a pas été altéré.
- Même si une modification est possible, c'est très peu probable en pratique.

Minage

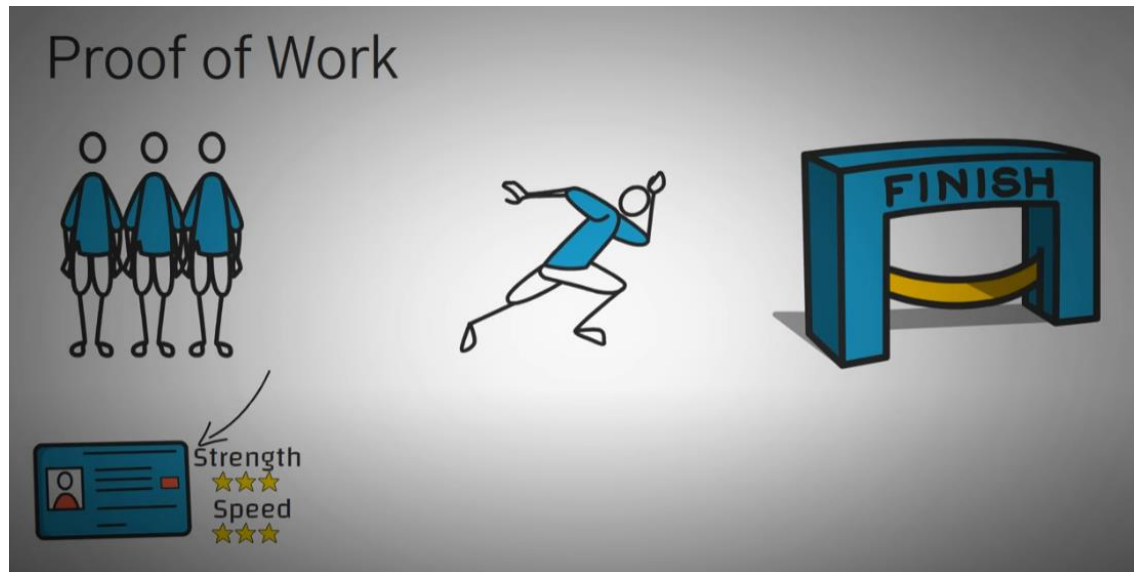
- Lorsque quelqu'un trouve une preuve de travail satisfaisante, il reçoit une récompense (inflation)
 - 6,25 bitcoins
- Cependant, cela est très rare. Sachant que, pour être valide, un hash doit commencer par un très grand nombre de zéros, il faut calculer la fonction de hachage de nombreuses fois, ce qui peut prendre des milliers d'années sur un rig de faible capacité.
 - D'où groupement de mineurs pour augmenter la probabilité, et partage des récompenses (en fonction du nombre de « shares »).
- Miner est-il intéressant?
 - Aujourd'hui, peut-être, mais soit en groupe, soit avec une ferme à bitcoin.
 - La récompense est divisée en deux tous les 210,000 blocs. Avant le 11 mai 2020, miner un bloc rapportait 12,5 bitcoins.
 - Prochain *halving* estimé en 2024

Les problèmes à résoudre

- Dans ce type de base de données, on ne fait qu'ajouter de l'information.
 - La blockchain grandit d'environ 50Go chaque année (bitcoin), et atteint aujourd'hui environ 430 Go
 - Comment repartir d'une chaîne plus petite tout en assurant la sécurité?
 - Solution jusqu'ici : augmenter la taille des blocs pour réduire le caractère exponentiel de l'évolution.
- Quid de la protection de la vie privée?
 - Fonctionne très bien avec bitcoin, car l'argent ne transite que d'un compte à l'autre, mais on ne sait pas à qui appartient le compte.
 - Pour d'autres types de données (ex. médicales), les informations apparaîtraient en clair.
- Pour bitcoin : Le nombre de bitcoins à miner est fini (21 millions de bitcoins).
 - Le dernier bitcoin sera miné aux alentours de l'année 2140.(mais reste environ 10% à miner)
 - Comment dès lors inciter les gens à continuer à calculer les hashes?
 - Inflation non contrôlée (Ethereum), mais valeur monnaie?

Problème en voie de résolution

- Coût énergétique du minage >>
- Proof-of-work = chacun pour soi



- Celui qui trouve la réponse gagne
- Les autres ont dépensé leurs ressources pour rien

Staking

- Proof-of-stake



Staking (2)

- Placement d'un « stake », une « mise », un « pari »
 - ≥ 32 ETH
 - Pool de staking possible
- Consensus de staking (algorithme)
 - Basé sur plusieurs critères : montant du stake, temps d'attente, valeur aléatoire, ...
 - Seul ce nœud est choisi pendant la phase de consensus
- Après vérification du nœud (par 128 validateurs sur Ethereum)
 - Si bloc validé $\rightarrow$ reward (comme sur proof-of-work)
 - Si bloc invalide $\rightarrow$ pas de reward
 - Dans les deux cas, stake consommé

Avantages/inconvénients

- Avantages

- Pas de conso d'énergie inutile (le mineur est choisi et il n'y a que lui qui mine le bloc)
- Sécurité équivalente (le bloc est toujours validé)
 - Attention, seulement 128 validateurs, donc attaque par majorité toujours possible, mais coûterait très cher.
- Plus facile d'augmenter la vitesse d'ajout d'un bloc sans en altérer la sécurité (ne dépend pas d'une puissance de calcul)

- Inconvénients

- Besoin d'être dispo quand choisi
 - Machine défaillante = perte du stake
 - Possibilité de déléguer à un validateur tiers (fiabilité?)
- Dans certaines cryptos, le stake est verrouillé et impossible de le récupérer
- Rendement sur le long terme et pas garanti
- Perte possible : « impermanent loss »
 - Ex : Gain de 15% en ETH
 - Ex: Valeur monétaire de l'ETH en perte de 50%

Table des matières

- Blockchain
- Bdd orientées objet
- NoSQL

Motivation

Le modèle relationnel connaît un très grand succès et s'avère très adéquat pour les applications traditionnelles des bases de données (gestion).

Il est beaucoup moins adapté aux nouvelles applications plus complexes telles que :

- CAD/CAM (Computer-Aided Design/Computer-Aided Manufacturing)
- BD d'images et de graphiques
- BD géographiques (GIS : Geographic Information Systems)
- BD multimédia (son, image, texte, etc. combinés)

Motivation (2)

Une caractéristique importante des BD orientées objet est qu'elles donnent au concepteur de la BD la capacité de spécifier

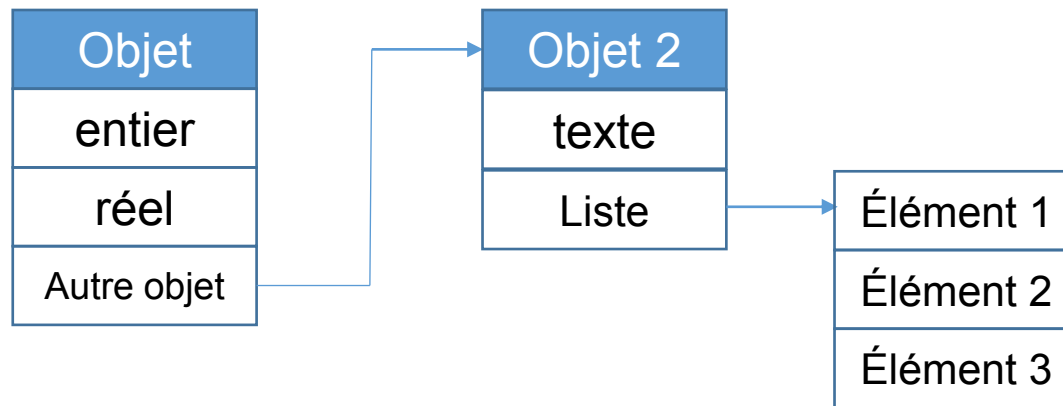
- non seulement la structure d'objets complexes,
- mais aussi les opérations à appliquer à ces objets.

Oracle permet la représentation de tels objets.

Ce système reprend les concepts adoptés dans les langages de programmation orientés-objet, avec les spécificités des systèmes de bases de données (persistance des données, transactions, etc.).

Les bases de données orientées-objet : pourquoi?

- Structures d'objets complexes:



Exemple d'objet (tuple)

- Une personne est un objet tuple de type

$$T_{pers} = \{(NOM, D_{string}), \\ (PRENOM, D_{string}), \\ (DATE_NAISS, D_{date}), \\ (ENFANTS, T_{enfants}), \\ (ADRESSE, T_{adresse})\}.$$

Avec le type $T_{adresse} =$

$$\{(RUE, D_{string}), (NUM, D_{int}), (CP, D_{int}), (LOC, D_{string})\}$$

Et le type $T_{enfants} = 2^{D_{string}}$

Schéma réseau routier

Pour définir les routes en tant qu'objets complexes, on considère un ensemble de domaines de base contenant

- D_{lat} : le domaine des latitudes,
- D_{long} : le domaine des longitudes,
- D_{id} : le domaine des identifiants de routes,
- D_{cat} : le domaine des catégories,
- D_{aut} : le domaine des autorités.

Un réseau routier est représenté par des objets de la manière suivante.

- Un point d'intersection est un objet tuple de type

$$T_{point} = \{(LATITUDE, D_{lat}), (LONGITUDE, D_{long})\}.$$

- Un segment est un objet tuple de type

$$T_{seg} = \{(POINT1, T_{point}), (POINT2, T_{point}), (CATEGORIE, D_{cat})\}.$$

- Une liste de segments est un objet ensemble de type $2^{T_{seg}}$.

- Une route est un objet tuple de de type

$$T_{route} = \{(ID_R, D_{id}), (LISTE_SEG, 2^{T_{seg}}), (AUTORITE, D_{aut})\}.$$

Caractéristiques

Une base de données orientée objet doit avoir un modèle possédant les caractéristiques suivantes :

- objets complexes,
- identificateurs d'objets,
- héritage,
- encapsulation,
- liaison tardive.

Combiner modèle relationnel et objet?

- Le modèle objet permet de traiter des objets complexes et donne de la flexibilité au niveau des types de données possibles.
- Le modèle relationnel est associé à un langage d'interrogation de haut niveau et permet l'exécution efficace de requêtes.
- Combiner les deux modèles permettrait de bénéficier des avantages des deux approches.
- On trouve de plus en plus des extensions “orientées-objet” des modèles relationnels ; SQL-99 prévoit de telles extensions.

Oracle implémente cette extension.

Création d'un type objet

- Ressemble à la création d'une table
- En plus des membres classiques, on peut y rajouter des procédures et des fonctions membres

```
-- Création d'un type objet
CREATE TYPE person_typ AS OBJECT (
  idno NUMBER,
  first_name VARCHAR2(20),
  last_name VARCHAR2(25),
  email VARCHAR2(25),
  phone VARCHAR2(20),
  MAP MEMBER FUNCTION get_idno RETURN NUMBER,
  MEMBER PROCEDURE display_details ( SELF IN OUT NOCOPY person_typ )
);
```

- Note : « MAP » signifie que la valeur de retour pourra être utilisée dans le calcul d'une expression

Définition des fonctions

- Un peut à la manière des packages, il faut définir le comportement des procédures/fonctions

```
-- Création du body correspondant (pour les fonctions et procédures)
CREATE TYPE BODY person_typ AS
  MAP MEMBER FUNCTION get_idno RETURN NUMBER IS
  BEGIN
    RETURN idno;
  END;
  MEMBER PROCEDURE display_details ( SELF IN OUT NOCOPY person_typ ) IS
  BEGIN
    -- use the PUT_LINE procedure of the DBMS_OUTPUT package to display details
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(idno) || ' ' || first_name || ' ' || last_name);
    DBMS_OUTPUT.PUT_LINE(email || ' ' || phone);
  END;
END;
```

Utilisation des types dans des tables

- Création et population d'une table utilisant ce type objet :

```
-- Création d'une table utilisant ces types
```

```
CREATE TABLE contacts (  
    contact person_typ,  
    contact_date DATE  
);
```

```
-- Insertion d'une valeur
```

```
INSERT INTO contacts VALUES (  
person_typ (65, 'Verna', 'Mills', 'vmills@example.com', '1-650-555-0125'),  
to_date('24 Juin 2003', 'dd Mon YYYY'));
```

Appel des méthodes

```
-- Récupération du numéro (de deux façons différentes)
SELECT c.contact.get_idno() FROM contacts c;
/
DECLARE
    p person_typ;
BEGIN
    SELECT c.contact INTO p
    FROM contacts c
    WHERE c.contact.get_idno() = 65;
    p.display_details();
END;
```

Le type REF

- Permet de rajouter une **référence** à un objet à l'intérieur d'un objet.
- Sorte de « pointeur d'objet ».
- Une référence circulaire est possible
 - Avoir une référence du même type d'objet
- Exemple :

```
-- Type REF dans la création d'objet
CREATE TYPE emp_person_typ AS OBJECT (
    name VARCHAR2(30),
    manager REF emp_person_typ
);
```

Table et population (REF)

```
-- Une table correspondante
CREATE TABLE emp_person_obj_table OF emp_person_typ;

-- Insertion de données
INSERT INTO emp_person_obj_table VALUES (
    emp_person_typ ('John Smith', NULL));

INSERT INTO emp_person_obj_table
    SELECT emp_person_typ ('Bob Jones', REF(e))
    FROM emp_person_obj_table e
    WHERE e.name = 'John Smith';
```

Déréférencement

- Suivre la référence pour en récupérer le contenu (DEREF)

```
-- Déréférencement implicite
select e.manager.name from emp_person_obj_table e;

-- Déréférencement explicite
select deref(e.manager).name from emp_person_obj_table e;
```

Héritage

- Il est possible de faire hériter un type d'objet à partir d'un autre type d'objet
- Les méthodes peuvent également être redéfinies.
- **! Pas d'héritage multiple !**
- Exemple de type d'objet parent :

```
-- Pour montrer l'héritage (NOT FINAL = peut être dérivée)
CREATE OR REPLACE TYPE Tpersonne AS OBJECT
(
  AVS CHAR(11),
  nom VARCHAR(20) ,
  prenom Tprenoms,
  conjoint REF TPersonne,
  MEMBER FUNCTION show RETURN VARCHAR2
) NOT FINAL;

CREATE OR REPLACE TYPE BODY Tpersonne AS
MEMBER FUNCTION show RETURN VARCHAR2 IS
BEGIN
  RETURN 'ID: ' || AVS || ', Nom: ' || nom || ', Prénom: ' || prenom(1);
END;
END;
```

Le type dérivé

```
-- Type dérivé
CREATE TYPE Tetudiant UNDER Tpersonne
(
    faculte VARCHAR(18),
    cycle VARCHAR(18),
    OVERRIDING MEMBER FUNCTION show RETURN VARCHAR2
);

CREATE TYPE BODY Tetudiant AS
    OVERRIDING MEMBER FUNCTION show RETURN VARCHAR2 IS
BEGIN
    RETURN (self AS Tpersonne).show || ' -- faculte: ' || faculte || ', cycle: ' || cycle ;
END;
END;
```

Généralisation

- Il est possible:
 - D'ajouter un objet dérivé dans une table d'objet parents
 - Attention, les spécificités des objets dérivés seront perdues!
 - D'appeler la version « parent » d'une méthode avec du casting

```
-- Des tables utilisant ces types
CREATE TABLE LesPersonnes OF Tpersonne;
CREATE TABLE LesEtudiants OF Tetudiant;

-- On peut, bien sûr, insérer dans chacune des tables
INSERT INTO LesPersonnes VALUES(11111, 'Rochat', Tprenoms('Philippe'), NULL);
INSERT INTO LesEtudiants VALUES(22222, 'Muller', Tprenoms('Annie', 'Marie'), NULL, 'HEC', 'Master');

-- Généralisation
INSERT INTO LesPersonnes VALUES(Tetudiant(22222, 'Muller', Tprenoms('Annie', 'Marie'), NULL, 'HEC', 'Master'));
COMMIT;

SELECT * FROM LesPersonnes; -- récupère les personnes (attention que les valeurs spécifiques des étudiants sont perdues)
SELECT * FROM LesEtudiants; -- récupère les étudiants
SELECT * FROM LesPersonnes p WHERE VALUE(p) IS OF (Tetudiant); -- récupère les personnes qui sont des étudiants

-- Invocation généralisée
DECLARE
myvar Tetudiant := Tetudiant(22222, 'Muller', Tprenoms('Annie', 'Marie'), NULL, 'HEC', 'Master');
BEGIN
  DBMS_OUTPUT.PUT_LINE(myvar.show);
  DBMS_OUTPUT.PUT_LINE((myvar AS Tpersonne).show); -- appel de la méthode de la classe parent
END;
```

Table des matières

- Blockchain
- Bdd orientées objet
- NoSQL

Succès des BDD relationnelles

- Modèle solide et bien standardisé.
- Technologie bien au point et implémentée efficacement.
- Gestion des transactions offrant des garanties fortes (ACID).
- Permet l'intégration de données en servant de support unique à plusieurs applications.

La contestation

Entreprises émergentes (Amazon, Google, Linkedin, ...) avec des besoins particuliers :

- Très vastes quantités de données (petabyte 10^{15} , hexabyte 10^{18} , zettabyte 10^{21} , ...),
- Utilisation de très grandes grappes (*clusters*) de machines,
- Besoin de rapidité face à un usage intense et fiabilité par réplication pour des données dont le schéma n'est pas figé,
- Coût très élevé des systèmes relationnels.

Développement de nouveaux systèmes de gestion de données NoSQL (Not, Not only SQL)

Que sont les BDD NoSQL

Modèle de données

- Données organisées en agrégats.
- Un agrégat est un ensemble des données que l'on consulte habituellement ensemble, par exemple l'information concernant un client et ses commandes.
- On accède à un agrégat à partir d'une clé.
- Les données sont donc vues comme des ensembles de paires (*clé, valeur*) où les *valeur* peut avoir un schéma plus ou moins complexe, fixé ou non.

Implémentation

Tables *hash* distribuées:

- Les paires (*clé*, *valeur*) sont réparties sur les différentes machines d'une grappe. On utilise le terme *sharding* (éclatement).
- Les mêmes données peuvent apparaître plusieurs fois pour assurer la fiabilité et la rapidité d'accès.
- Mais, se pose alors le problème de la cohérence: que se passe-t-il lors d'une mise à jour?

Transactions

La référence dans le modèle relationnel est la transaction ACID (Atomicité, Cohérence, Isolation, Durabilité).

Dans les bases de données NoSQL l'exigence de transactions ACID est abandonnée au profit de transactions BASE:

- ***Basically Available*** (la disponibilité est assurée),
- ***Soft-state*** (l'état du système peut évoluer, même sans opérations effectuées),
- ***Eventually consistent*** (la cohérence est atteinte à terme).

NoSQL vs Relationnel

Dans le relationnel les données doivent être recherchées dans plusieurs tables. Comment les répartir dans une grappe?

Les transactions ACID deviennent très coûteuses à garantir dans un contexte de réplication. Il faut synchroniser toutes les copies.

Les garanties fortes offertes par les transactions du relationnel ne sont pas toujours nécessaires.

NoSQL privilégie un temps de réponse court pour tous les utilisateurs par rapport à une cohérence parfaite.

Utiliser des données pas (ou moins) structurées a ses avantages.

NoSQL pour tous?

NON

- La cohérence est souvent indispensable.
- Il n'y a pas de modèle standardisé pour NoSQL, d'où l'existence d'une multitude de systèmes incompatibles.
- NoSQL est mal adapté aux situations où les mêmes données sont utilisées dans des applications ayant des besoins différents.
- NoSQL a ses applications, mais les systèmes relationnels restent largement les plus utilisés.

La plus connue : MongoDB

- Base de données NoSQL la plus connue et la plus utilisée
- Gratuite
- Stocke les données sous forme de documents JSON

```
{  
  "_id": ObjectId("5ce45d7606444f199acfba1e"),  
  "name": {given: "Alex", family: "Smith"},  
  "email": "email@example.com"  
  "age": 27  
}
```

- Supporte le sharding (base de données distribuée)

MongoDB

- Création DB
 - Use <nom>
- Lister les DB
 - show dbs
- Créer une collection (=~ table en SQL)
 - db.createCollection(<nom>,<options>)
- Lister les collections
 - show collections

MongoDB démo (2)

- Ajouter un document
 - `db.<collection>.insert(<document>)`
- Chercher un document
 - `db.<collection>.find()`
 - `db.<collection>.findOne()`
- Belle mise en page
 - `<...>.find().pretty()`
 - `findOne` est déjà « pretty »
- Critère de filtre (~= WHERE de SQL)
 - Dans `find()`, spécifier valeur (ou `$lt`, `$gt`, `$ne`, ...)

MongoDB démo (3)

- Modifier un document
 - `db.<collection>.update(<objet>,{ $set: <modif> })`
- Projection
 - `db.<collection>.find({<critère de recherche>},{<clé>:<1 ou 0>})`
- Tri
 - `db.<collection>.find().sort(<attribut>:<1 ou -1>)`
- Limite, offset
 - `(...).find().limit(<nb à afficher>).skip(<nb à passer>)`
- Groupements
 - `db.mycol.aggregate([{$group : { _id : "$nom", num_pers : {$sum : 1}}})`

MongoDB démo (4)

- Supprimer un document
 - `db.<collection>.remove(<critère>)`
- Supprimer une collection
 - `db.<collection>.drop()`
- Supprimer une base de données
 - `db.dropDatabase()`

MongoAPI

- Permet de se connecter à MongoDB via JDBC
- 2 packages :
 - bson-4.4.1.jar
 - mongo-java-driver-3.12.10.jar
- Vraiment très complet
 - Voir la doc

Example

```
public class TestMongoAPI {  
    public static void main(String[] args) {  
        // Replace the uri string with your MongoDB deployment's connection string  
        String uri = "mongodb://localhost:27017/?maxPoolSize=20&w=majority";  
        try (MongoClient mongoClient = MongoClient.create(uri)) {  
            MongoDBDatabase database = mongoClient.getDatabase("test");  
  
            try {  
                Bson command = new BsonDocument("ping", new BsonInt64(1));  
                Document commandResult = database.runCommand(command);  
                System.out.println("Connected successfully to server.");  
            } catch (MongoException me) {  
                System.err.println("An error occurred while attempting to run a  
command: " + me);  
            }  
        }  
    }  
}
```

Example (suite)

```
MongoCollection<Document> collection = database.getCollection("test");
List<String> list = new ArrayList<>();
//collection.find().map(Document::toJson).forEach(list::add);
try (MongoCursor<Document> cursor = collection.find().iterator()) {
    while (cursor.hasNext()) {
        System.out.println(cursor.next());
    }
}

Document to_add = new Document();
to_add.append("name", "Pierre Tom");
to_add.append("age",22);
collection.insertOne(to_add);
}
}
}
```